

אוניברסיטת בן-גוריון בנגב
Ben-Gurion University of the Negev

Hatchala - Introduction to computer science in Java - Fall 26'-27'

Department of Computer Science and Information

Ben-Gurion University of the Negev

Based on both Dr. Assaf Zaritsky slides for "Intro to CS" (IS)
and slides for "Introduction to CS" (CS) courses at BGU

Lecture 2: Loops and Arrays

Yesterday (Day 1):

- **Brief background to computation and programming**
- Java basics:
  - Variables
  - Arithmetic operators
  - Logic operators
  - Input/Output
  - Conditional statements

Using variables to store data

- A variable is a named storage location in a computer's memory that holds a value.
- Each variable has:
  - A value (content, the stored data)
  - A name (a shortcut to its address in memory)
  - A type (int, double, boolean, char)

Variables and assignments

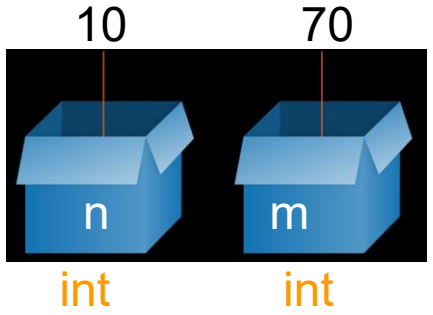
```
int n = 10;
int m = (10 + 4) * 5;
```

Left-hand side is a variable

Right-hand side is an expression

Java:

1. Evaluates the **expression**
2. Assigns its value to the **variable**



Variable's name - a sequence of letters and digits, starting with a letter

Arithmetic operators

Operator	Use	Description
+	x + y	Sum of x and y
-	x - y	Subtracts y from x
*	x * y	Multiplies x by y
/	x / y	Divides x by y
%	x % y	Modulo: the remainder of dividing x by y

Boolean (logical) operators

Operator		Description
a **&&** b	and	Both true: **true**, otherwise: **false**.
a **||** b	or	At least one is true: **true**, Otherwise: **false**.
! a	not	a is false: **true**, a is true: **false**

and

false	**true**	**&&**
false	true	**true**
false	false	**false**

or

false	**true**	**||**
true	true	**true**
false	true	**false**

not

	!
false	**true**
true	**false**

Comparison operators

Compares two numbers and returns a boolean literal

Operator	Name	Description
x < y	Less than	true if x is less than y, otherwise false.
x > y	Greater than	true if x is greater than y, otherwise false.
x <= y	Less than or equal to	true if x is less than or equal to y, otherwise false.
x >= y	Greater than or equal to	true if x is greater than or equal to y, otherwise false.
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

Comparison operators

Compares two booleans and returns a boolean literal

Operator	Name	Description
x == y	Equal	true if x equals y, otherwise false.
x != y	Not Equal	true if x is not equal to y, otherwise false.

Input / Output

Input: myScanner.nextInt();

Output:

System.out.print("Hello world");

System.out.println("Hello world");

```
1  import java.util.Scanner;
2  public class Main {
3
4      public static void main(String[] args) {
5
6          Scanner myScanner = new Scanner(System.in);
7          int n = myScanner.nextInt();
8          System.out.println(n);
9
10     }
11
12 }
```

import Scanner library containing all relevant data java requires

create a new "Scanner" object which can read input from the console

print the value of variable n to the console

Block statement

A block of code contains one or more statements inside curly brackets:

{
    statement1
    statement2
}

```
{
        System.out.println("prime");
        System.out.println("hurray!");
}
```

Each block statement is indented one “tab”.

Conditional statement: if

Used to conditionally execute a block of code

Syntax

if *(expression)* **{**
    statement1
    statement2
}

```
if (isPrime) {
     System.out.println("prime");
     System.out.println("hurray!");
}
```

If *expression* is true, *statement* are executed.

if is a reserved word in Java

Conditional statement: if else

Used to conditionally execute one of two blocks of code

Syntax

if *(expression)* **{**
    statement1
    statement2
}
else {
    statement3
    statement4
}

```
if (isPrime) {
    System.out.println("prime");
}
else {
    System.out.println("not prime");
}
```

If *expression* is true, the first block is executed. Else, the second block is executed.

else is a reserved word in Java

Complex boolean expressions

Not actually complex!

- Logical operators transform multiple Boolean values to a single Boolean value (nothing new)
- Let's break it down to bits and pieces:

true if **at least one** is negative

```
(a <= 0 || b <= 0 || c <= 0)
```

true if a or b are negative **and** c is negative

```
(((a <= 0) || (b <= 0)) && c <= 0)
```

Nested conditional statements = conditional statements within conditional statements

Not actually complex!

- Every code block is “stand alone”.
- if / if-else statements can be placed within the block of an if/ if/else statement (nothing new) >>
- Allows for more sophisticated flow control!

- Let’s break it down to pieces

Reminder: triangle inequality

```
System.out.print("Enter first edge: ");
double a = myScanner.nextDouble();

System.out.print("Enter second edge: ");
double b = myScanner.nextDouble();

System.out.print("Enter third edge: ");
double c = myScanner.nextDouble();

// Check that all edges are positive
if (a <= 0 || b <= 0 || c <= 0) {
    System.out.println("All edges must be positive!");
}

// Triangle inequality rule:
// The sum of any two sides must be greater than the third.
else if (a + b >= c && a + c >= b && b + c >= a) {
    System.out.println("Yes, these edges can form a triangle.");
}
else {
    System.out.println("No, these edges cannot form a triangle.");
}
```

"if" block

"else" block containing another if-else statement

ASCII table

- Encoding letters from assigned numbers

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL (null)	32	SPACE	64	@	96	`
1	SOH (start of heading)	33	!	65	A	97	a
2	STX (start of text)	34	"	66	B	98	b
3	ETX (end of text)	35	#	67	C	99	c
4	EOT (end of transmission)	36	$	68	D	100	d
5	ENQ (enquiry)	37	%	69	E	101	e
6	ACK (acknowledge)	38	&	70	F	102	f
7	BEL (bell)	39	'	71	G	103	g
8	BS (backspace)	40	(	72	H	104	h
9	TAB (horizontal tab)	41	)	73	I	105	i
10	LF (NL line feed, new line)	42	*	74	J	106	j
11	VT (vertical tab)	43	+	75	K	107	k
12	FF (NP form feed, new page)	44	,	76	L	108	l
13	CR (carriage return)	45	-	77	M	109	m
14	SO (shift out)	46	.	78	N	110	n
15	SI (shift in)	47	/	79	O	111	o
16	DLE (data link escape)	48	0	80	P	112	p
17	DC1 (device control 1)	49	1	81	Q	113	q
18	DC2 (device control 2)	50	2	82	R	114	r
19	DC3 (device control 3)	51	3	83	S	115	s
20	DC4 (device control 4)	52	4	84	T	116	t
21	NAK (negative acknowledge)	53	5	85	U	117	u
22	SYN (synchronous idle)	54	6	86	V	118	v
23	ETB (end of trans. block)	55	7	87	W	119	w
24	CAN (cancel)	56	8	88	X	120	x
25	EM (end of medium)	57	9	89	Y	121	y
26	SUB (substitute)	58	:	90	Z	122	z
27	ESC (escape)	59	;	91	[	123	{
28	FS (file separator)	60	<	92	\	124	|
29	GS (group separator)	61	=	93	]	125	}
30	RS (record separator)	62	>	94	^	126	~
31	US (unit separator)	63	?	95	_	127	DEL

Plan for today (Day 2):

- **Loops**
- Arrays

Code Flow

Loops

Flow control

- Conditional Statements R**educe executed code**. We write a lot, only a part gets executed

- Loops **Multiply executed code** –
  We write code, the interpreter executes it multiple times.

Motivation: Let's count down from 5!

Loops.java × Main.java Square.java

```
1   public class Loops {
2       public static void main(String[] ar
3           int n = 5;
4           System.out.println(n);
5           n = n - 1;
6           System.out.println(n);
7           n = n - 1;
8           System.out.println(n);
9           n = n - 1;
10          System.out.println(n);
11          n = n - 1;
12          System.out.println(n);
13      }
14  }
15
```

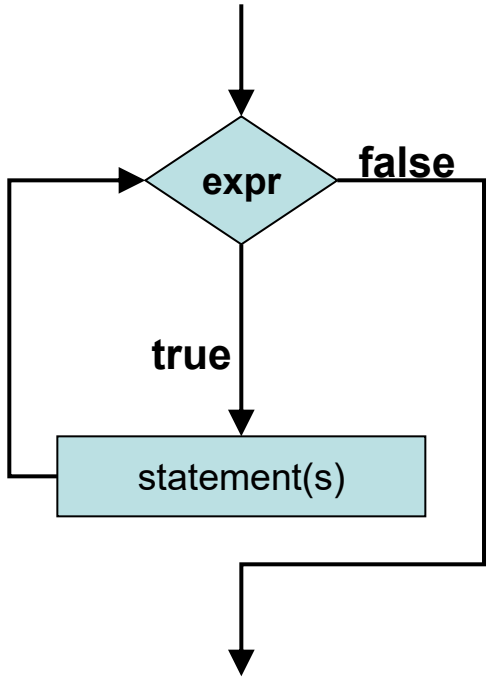
- Linear code
- Repeating structure
- How do we count **to 5**?



While loop

Used to repeat the same instructions until a stop criterion is met

```
while (boolean condition) {
    statement1
    statement2
    …
}
```



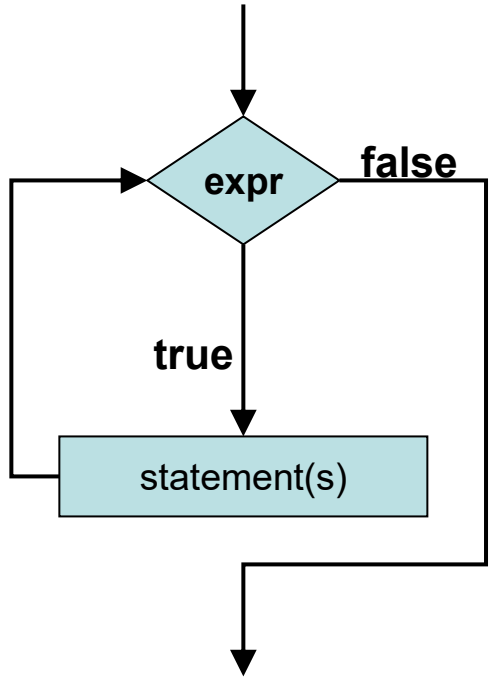
while is a *reserved word* in Java

Example: Count down from 5!



Example 1 – countdown

```
int a = 5;
while (a > 0){
    System.out.println(a);
    a = a - 1;
}
```



Loops

```
"C:\Program Files\Java\jdk-15.0.2\bin\java.exe" "-javaag
5
4
3
2
1

Process finished with exit code 0
```

Example 2 – factorial (עצרת)

Definition: n! = n*(n-1)*(n-2)*…*3*2*1 0!=1

Examples:

5! = 5*4*3*2*1

3! = 3*2*1



Read a non-negative number n and print n!

Example 2 – factorial (עצרת)

```
System.out.println("Enter a non-negative number");
int n = myScanner.nextInt();
int i = n;
int result = 1;
while(i > 0){
    result = result * i;
    i = i - 1;
}
System.out.println("n! = " + result);
```

Loops

```
"C:\Program Files\Java\jdk-15.0.2\bin\java.exe" "-javaagent:C:\Pr
Enter a non-negative number
5
n! = 120

Process finished with exit code 0
```

Example 3: smallest divisor

Input: a positive whole number n

Output: the smallest prime number which divides n

Examples:

- n = 6 smallest divisor = 2
- n = 35 smallest divisor = 5
- n = 221 smallest divisor = ?



Read a positive number n and print it's smallest divisor

Example 3: smallest divisor

```
System.out.println("Enter a non-negative number");
int m = myScanner.nextInt();

int devisor = 2;
while(m % devisor != 0){
    devisor = devisor + 1;
}
System.out.println("The smallest devisor of " + m + " is: " + devisor);
```

Will this while loop **always** return the smallest divisor for any positive number?

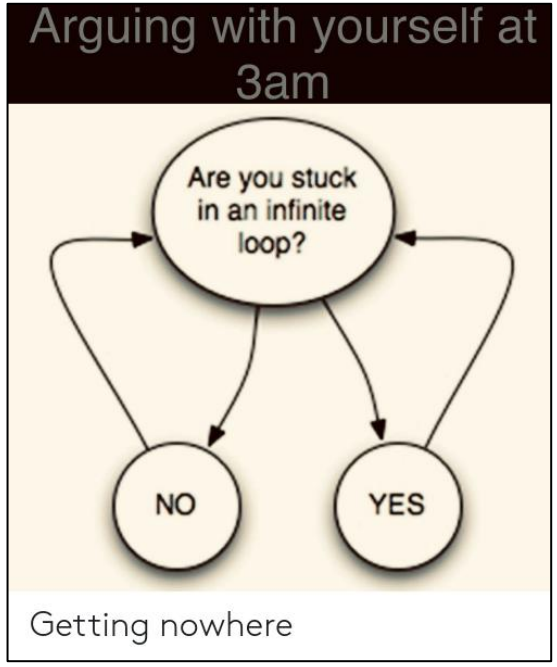
Now fix it… ☺

Loops ×

```
"C:\Program Files\Java\jdk-15.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrair
Enter a non-negative number
221
The smallest devisor of 221 is: 13

Process finished with exit code 0
```

(Beware of) infinite loops



"An infinite loop is a sequence of instructions in a computer program which loops endlessly, either due to the loop having no terminating condition, having one that can never be met, or one that causes the loop to start over." (Wikipedia)



Write a simple infinite loop



Example 3: smallest divisor: Fixing the problem

Input: a positive whole number n

Output: the smallest prime number which divides n

Examples:

- n = 6 smallest divisor = 2
- n = 35 smallest divisor = 5
- n = 221 smallest divisor = ?



Read a positive number n and print it's smallest divisor

Example 3: Smallest divisor (no infinite loops)

```
System.out.println("Enter a non-negative number");
int m = myScanner.nextInt();

if (m == 1){
    System.out.println("The smallest devisor of " + m + " is: " + m);
}
else{
    int devisor = 2;
    while(m % devisor != 0){
        devisor = devisor + 1;
    }
    System.out.println("The smallest devisor of " + m + " is: " + devisor);
}
```

Example 4: all divisors

Input: a positive whole number n

Output: all divisors of n



Examples:

- n = 6 divisors: 1, 2, 3, 6
- n = 50 divisors: 1, 2, 5, 10, 25, 50

Read a positive number n and print all it's divisors

Example 4: all divisors

```
        System.out.println("Enter a non-negative number");
        int number = myScanner.nextInt();
        int dev = number;
        while (dev > 0) {
            if(number % dev == 0){
                System.out.println(dev);
            }
            dev = dev - 1;
        }
    }
```

Loops

```
"C:\Program Files\Java\jdk-15.0.2\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\Intell
Enter a non-negative number
50
50
25
10
5
2
1
```

Example – addition

```
# Add the numbers from 1-to-10
1+2+3+4+5+6+7+8+9+10
```

```
55
```

- What if we want to sum the numbers 1-to-100 or 1,000 or 1,000,000?
- We can use while loops
- If we want to iterate over the integers in predefined sequential order there is another option – a for loop

for loop

```
for (<initialize>; <boolean condition>; <increment>) {
    statement1
    statement2
    …
}
```

```
for (int a = 0; a <= 5; a = a + 1) {
    System.out.println(a);
}
```

1) initialize
2) check boolean condition
3) if true:
    1) execute block
    2) increment>
    3) go back to step 2
4) else, exit loop and execute next line

for is a *reserved word* in Java

Example 4: print the sum of all even numbers up to a given positive number using a for loop

```
System.out.println("Enter a positive number");
int n = myScanner.nextInt();
int sum = 0;
for (int i = 2; i <= n; i = i + 2){
    sum = sum + i;
}
System.out.println(sum);
```



Loops

```
"C:\Program Files\Java\jdk-15.0.2\bin\java.exe" "-javaagent:C:\Program F
Enter a posotive number
9
20

Process finished with exit code 0
```

Fibonacci series

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …



Definition:

fib(0) = 0
fib(1) = 1
fib(n) = fib(n-1) + fib(n-2)

What is the next number in the series?

Fibonacci

Write a program that reads a positive number n and prints the n’th Fibonacci number.

Where do we start…???



First: write your “high level” algorithm

1. we
2. need
3. to
4. plan
5. ahead!

Fibonacci – high level algorithm

```
1. נקלוט מהמשתמש מספר חיובי n
2. נכריז על משתנה result
3. אם n = 1 נשים את הערך 0 או a במשתנה result
4. אם n = 2 נשים את הערך 0 או b במשתנה result
5. אחרת
    5.1 נאתחל משתנים a,b,c עם ערכים 0,1,0 בהתאמה
    5.2 נרוץ בלולאה על הערכים 3 עד n ובכל איטרציה נקדם את המשתנים:
            c ← a + b
            a ← b
            b ← c
    5.3 נציב את הערך של b במשתנה result
6.נדפיס את התוצאה השמורה ב result
```

Fibonacci – high level algorithm

```
1. נדפיס למסך בקשה להקלדת מספר חיובי n
2. נקלוט את המספר למשתנה בשם n
3. אם לא התקבל מספר חיובי נדפיס הודעה מתאימה
4. אחרת,
    4.1 נכריז על משתנה result
    4.2 אם n = 1 נשים את הערך 0 במשתנה result
    4.3 אם n = 2 נשים את הערך 0 במשתנה result
    4.4 אחרת
        4.4.1 נאתחל משתנים a,b,c עם ערכים 0,1,0 בהתאמה
        4.4.2 נרוץ בלולאה על הערכים 3 עד n
              ובכל איטרציה נקדם את המשתנים:
                    c ← a + b
                    a ← b
                    b ← c
        4.4.3 נציב את הערך של b במשתנה result
5.נדפיס את התוצאה
```

Fibonacci – high level algorithm

```
1. Print "Enter a positive number n: "
   2. n ← a whole positive number entered by the user
   3. if (n <= 0)
        Print "Please enter a positive integer. Try again."
   4. else
                result
                if (n = 1)
                        result ← 0
                else if (n = 2)
                        result ← 1
                else
                        a = 0, b = 1, c = 0
                        for i = 3 to n do
                                c ← a + b;
                                a ← b;
                                b ← c;
                        result = b;
5. Print "The " + n + "th Fibonacci number is: " + result
```

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer. Try again.");
}
else {
```

We can start calculating

```
}
```

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer. Try again.");
}
else {
    int result;
    if (n == 1) {
        result = 0;
    }
    else if (n == 2) {
        result = 1;
    }
    else {
        for (int i = 3; i <= n; i = i + 1) {
            c = a + b;
            a = b;
            b = c;
        }
        result = b;
    }
    System.out.println("The " + n + "th Fibonacci number is: " + result);
}
```

Initialize a variable named **result** and print it

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer. Try again.");
}
else {
    int result;
    if (n == 1) {
        result = 0;
    }
    else if (n == 2) {
        result = 1;
    }
    else {
        int a = 0, b = 1, c = 0;
        for (int i = 3; i <= n; i = i + 1) {
            c = a + b;
            a = b;
            b = c;
        }
        result = b;
    }
    System.out.println("The " + n + "th Fibonacci number is: " + result);
}
```

Handle base cases

Initialize three loop variables and iterate

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer. Try again.");
}
else {
    int result;
    if (n == 1) {
        result = 0;
    }
    else if (n == 2) {
        result = 1;
    }
    else {
        int a = 0, b = 1, c = 0;
        for (int i = 3; i <= n; i = i + 1) {
            c = a + b;
            a = b;
            b = c;
        }
        result = b;
    }
    System.out.println("The " + n + "th Fibonacci number is: " + result);
}
```

Initialize three loop variables and iterate

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer. Try again.");
}
else {
    int result;
    if (n == 1) {
        result = a;
    }
    else if (n == 2) {
        result = b;
    }
    else {
        int a = 0, b = 1, c = 0;
        for (int i = 3; i <= n; i = i + 1) {
            c = a + b;
            a = b;
            b = c;
        }
        result = b;
    }
    System.out.println("The " + n + "th Fibonacci number is: " + result);
}
```

Iterate

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer. Try again.");
}
else {
    int result;
    if (n == 1) {
        result = 0;
    }
    else if (n == 2) {
        result = 1;
    }
    else {
        int a = 0, b = 1, c = 0;
        for (int i = 3; i <= n; i = i + 1) {
            c = a + b;
            a = b;
            b = c;
        }
        result = b;
    }
    System.out.println("The " + n + "th Fibonacci number is: " + result);
}
```

Iterate

Basic data structure in Java: array

Array - מערך

- הגדרה: מערך – רצף של תאים בזיכרון מטיפוס מסוים
- הקצאה: משפט new

```
int[] array = new int[3];
```

- תחביר דומה עבור מערכים של כל הטיפוסים

```
boolean[] array = new boolean[3];
```

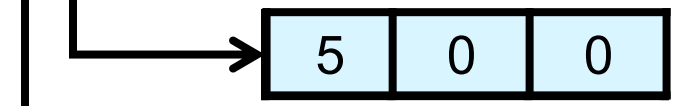
- ניתן להתייחס לערכים שבתאי המערך כ-

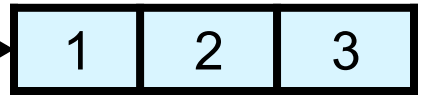
array[0], array[1], array[2],…

ייצוג מערך בזכרון – Array

name	type	Value
array1	int[]	
array2	int[]	

```
int[] array1;
array1 = new int[3];
array1[0] = 5;
int[] array2 = {1,2,3};
```



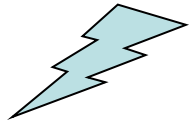


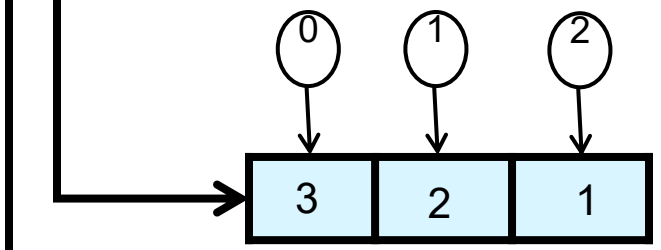


ייצוג מערכים בזיכרון

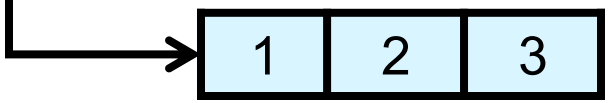
name	type	Value
array1	int[]	
array2	int[]	

```
array1[0] = 3;
array1[1] = 2;
array1[2] = 1;
array1[3] = 0;
```





אינדקסים

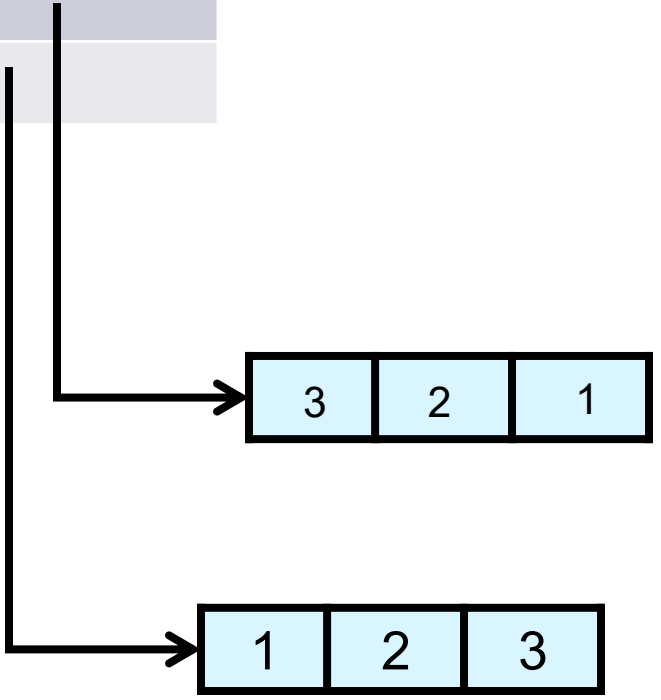




ייצוג מערכים בזיכרון

name	type	Value
array1	int[]	
array2	int[]	

```
array2 = array1;
```

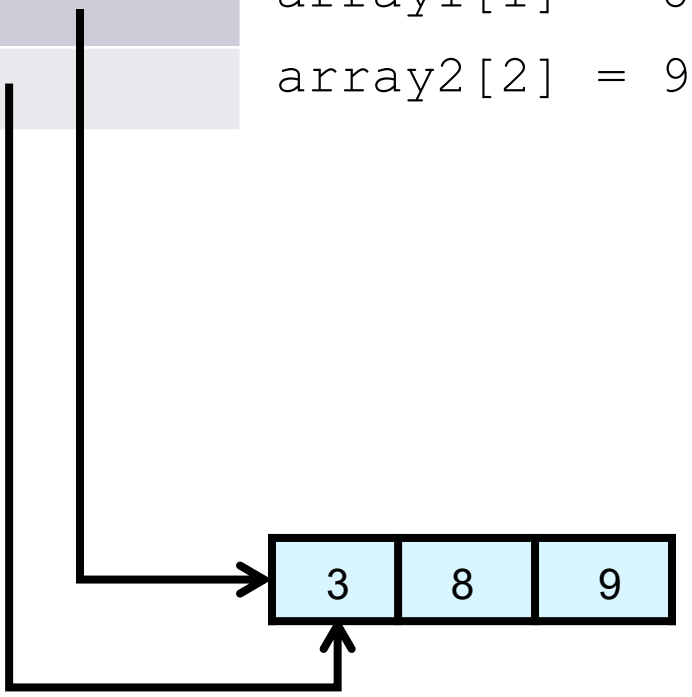




ייצוג מערכים בזיכרון

name	type	Value
array1	int[]	
array2	int[]	

```
array2 = array1;
array1[1] = 8;
array2[2] = 9;
```





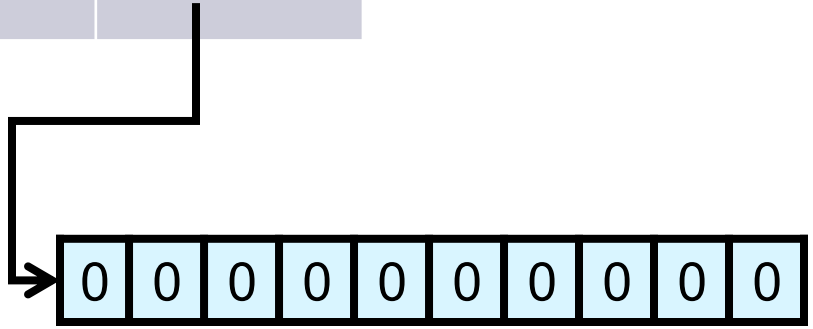


המאפיין length

- length הוא מאפיין (attribute) של מערך.
- במערך arr שהוקצה עבורו מקום בזיכרון מספר האיברים הוא arr.length:

```
int [] arr2 = new int[10];
System.out.println(arr2.length); // יודפס 10
```

name	type	Value
arr2	int[]	



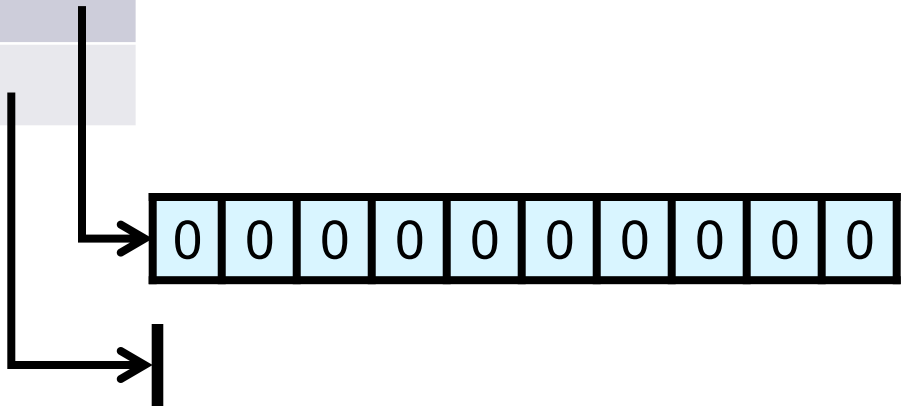


המאפיין length

- מספר האיברים יכול להיות אפס:

```
int [] arr3 = new int[0];
System.out.println(arr3.length); // יודפס 0
```

name	type	Value
arr2	int[]	
arr3	int[]	





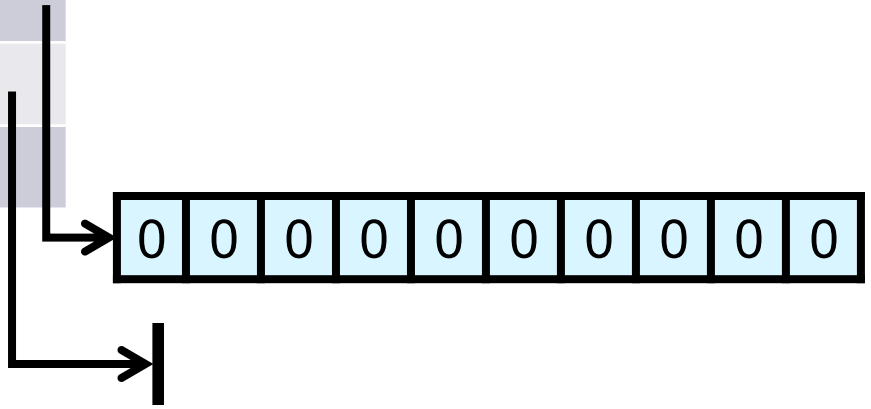
הערך המיוחד null

- ניתן גם לציין שהמערך אינו מצביע לאף כתובת (כלומר לא הוקצה עבורו מקום בזיכרון) ע"י שימוש ב-null:

```
int [] arr4 = null;
System.out.println(arr4.length);
```

Exception in thread "main" java.lang.NullPointerException at Test.main(Test.java:6)

name	type	Value
arr2	int[]	
arr3	int[]	
arr4	int[]	null



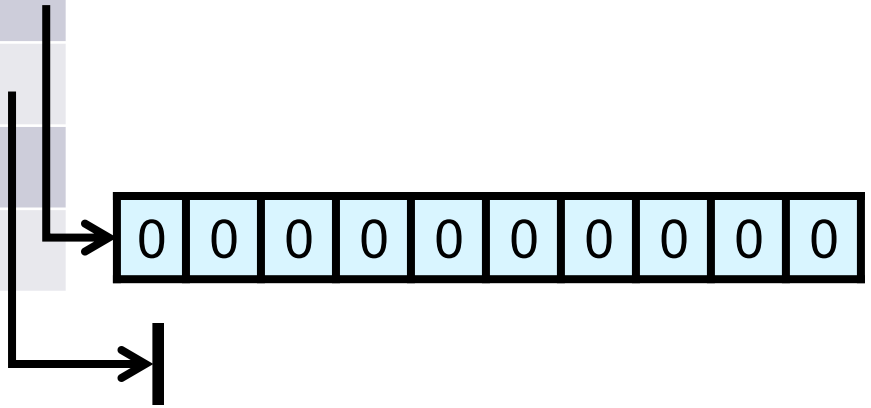


uninitialization

- אפשר לאתחל משתנה בלי להציב לתוכו ערך, אבל אז הניסיון להשתמש בו יכשל בזמן הקומפילציה

```
int [] arr5;
System.out.println(arr5.length);
```

name	type	Value
arr2	int[]	
arr3	int[]	
arr4	int[]	null
arr5	int[]	



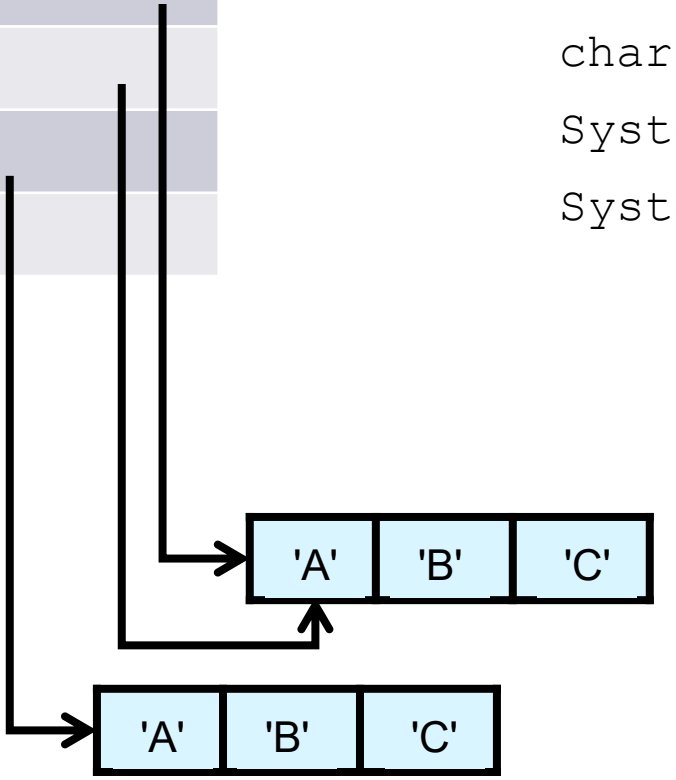


שוויון מערכים

name	type	Value
cArray1	char[]	
cArray2	char[]	
cArray3	char[]	

```
char[] cArray1= {'A', 'B', 'C'};
char[] cArray2= cArray1;
char[] cArray3= {'A', 'B', 'C'};
System.out.println(cArray1 == cArray2);
System.out.println(cArray1 == cArray3);
```





Example: Create an array and print each value in a new line




```
boolean[] a = {true, false};
for(int i = 0; i < a.length; i = i+1)
      System.out.println(a[i]);
```

לולאה זו מאפשרת לבצע מעבר סידרתי על אברי מערך בעזרת כתיב מקוצר.
לולאת for-each (במקרה של מערך) מתאימה למצבים שבהם רוצים לעבור על **כל** אברי המערך (לא ניתן להציב תנאי להמשך ריצה בלולאה).

שני קטעי הקוד הללו שקולים:

```
boolean[] a = {true, false};
for(boolean element : a)
      System.out.println(element);
```

Example:

Create an array of numbers and print the sum of all values




```
int[] b = {1,2,3};
int sum = 0;
for(int i = 0; i < b.length; i = i+1)
      sum = sum + b[i];
System.out.println(sum);
```

```
int[] b = {1,2,3};
int sum = 0;
for(int element : b)
      sum = sum + element;
System.out.println(sum);
```

Example:

Create an array of numbers and print the smallest value




```
double[] c = {1, -2.3, 7.4};
double minElement = c[0];
for (int i = 0; i < c.length; i = i+1)
    if(c[i] < minElement)
        minElement = c[i];
System.out.println(minElement);
```

```
double[] c = {1, -2.3, 7.4};
double minElement = c[0];
for (double element : c)
    if(element < minElement)
        minElement = element;
System.out.println(minElement);
```



בדיקת שוויון מערכים

בהנתן שני מערכים עבורם הוקצה מקום בזכרון, נרצה לבדוק האם הם מכילים את אותם האיברים ובאותו הסדר. נבדוק זאת בלולאה, איבר איבר.

```
int[] a = {1,2,3};
int[] b = {1,2,3};
boolean isEqual = (a.length == b.length); // isEqual=true iff |a|=|b|
for (int i = 0; i < a.length && isEqual; i = i + 1){
      if (a[i] != b[i]){
            isEqual = false;
      }
}
System.out.println(isEqual);
```

- בבית – "הרצה על הנייר"
- מה הם מקרי בדיקה מעניינים עבור הבעיה?



מערכים - סיכום

- מערכים מאפשרים לעבוד עם מספר גדול (ולא בהכרח ידוע מראש) של משתנים מאותו טיפוס.
- מערכים הם הטיפוסים המורכבים (לא פרימיטיביים) הראשונים שאנחנו פוגשים
  - למשתנה המפנה למערך יכול להיות ערך null.
  - רצף תאי המערך נמצא בזכרון.
  - בטבלת המשתנים נמצא הערך שלו : ההפניה אליו.

Reserved words in Java in our course

(you already know 18/31 of them ☺)

abstract	double	for	int	private	static
boolean	else	if	interface	protected	super
byte	extends	implements	long	public	this
char	final	import	new	return	throw
class	float	instanceof	package	short	void
					while

Summary – lecture #2

- Loops
- Arrays



Hatchala - Introduction to computer science in Java - Fall 25'-26'

Department of Computer Science and Information

Ben-Gurion University of the Negev

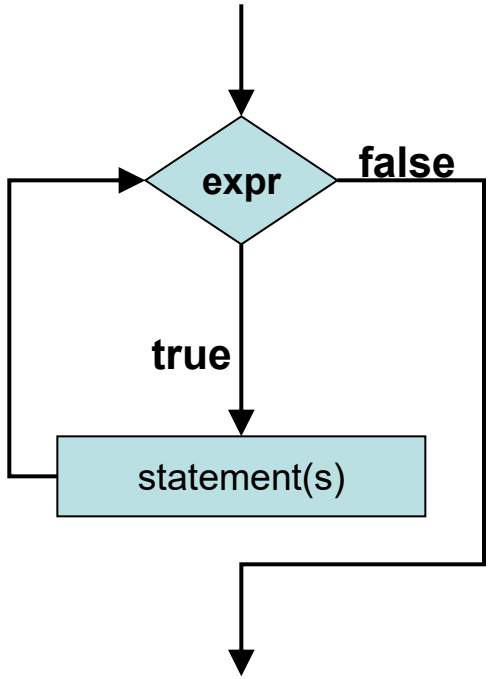
Based on both Dr. Assaf Zaritsky slides for "Intro to CS" (IS) and slides for "Introduction to CS" (CS) courses at BGU

Practical Session 2: Loops and Arrays

While loop

Used to repeat the same instructions until a stop criterion is met

```
while (boolean condition) {
    statement1
    statement2
    …
}
```



while is a *reserved word* in Java

Example 3: Smallest divisor (no infinite loops)

```
System.out.println("Enter a non-negative number");
int m = myScanner.nextInt();

if (m == 1){
    System.out.println("The smallest devisor of " + m + " is: " + m);
}
else{
    int devisor = 2;
    while(m % devisor != 0){
        devisor = devisor + 1;
    }
    System.out.println("The smallest devisor of " + m + " is: " + devisor);
}
```

for loop

```
for (<initialize>; <boolean condition>; <increment>) {
    statement1
    statement2
    …
}
```

```
for (int a = 0; a <= 5; a = a + 1) {
    System.out.println(a);
}
```

1) initialize
2) check boolean condition
3) if true:
    1) execute block
    2) increment>
    3) go back to step 2
4) else, exit loop and execute next line

for is a *reserved word* in Java

Fibonacci - code

```
Scanner myScanner = new Scanner(System.in);
System.out.print("Enter a positive number n: ");
int n = myScanner.nextInt();
if (n <= 0) {
    System.out.println("Please enter a positive integer.");
}
else {
    int a = 0, b = 1, c = 0;

    if (n == 1) {
        System.out.println("The " + n + "st Fibonacci number is: " + a);
    }
    else if (n == 2) {
        System.out.println("The " + n + "nd Fibonacci number is: " + b);
    }
    else {
        for (int i = 3; i <= n; i = i + 1) {
            c = a + b;
            a = b;
            b = c;
        }
        System.out.println("The " + n + "th Fibonacci number is: " + b);
    }
}
```

Array - מערך

- הגדרה: מערך – רצף של תאים בזיכרון מטיפוס מסוים
- הקצאה: משפט new

```
int[] array = new int[3];
```

- תחביר דומה עבור מערכים של כל הטיפוסים

```
boolean[] array = new boolean[3];
```

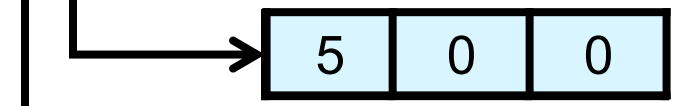
- ניתן להתייחס לערכים שבתאי המערך כ-

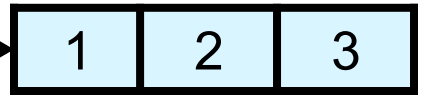
array[0], array[1], array[2],…

ייצוג מערך בזכרון – Array

name	type	Value
array1	int[]	
array2	int[]	

```
int[] array1;
array1 = new int[3];
array1[0] = 5;
int[] array2 = {1,2,3};
```





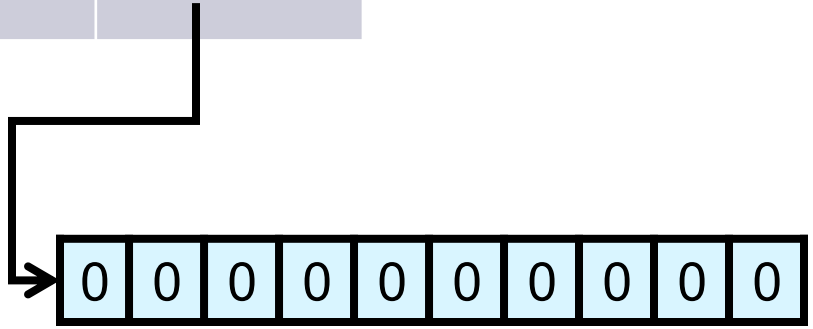


המאפיין length

- length הוא מאפיין (attribute) של מערך.
- במערך arr שהוקצה עבורו מקום בזיכרון מספר האיברים הוא arr.length:

```
int [] arr2 = new int[10];
System.out.println(arr2.length); // יודפס 10
```

name	type	Value
arr2	int[]	



מערכים - תרגיל חימום

Write a Java statement creating each array:



name	type	Value
arr1	int[]	
arr2	int[]	
arr3	int[]	
arr4	int[]	null
arr5	int[]	

